

Objectives :

- Build a virtual network and verify connectivity (Lab0)
- Configure and verify Static/Dynamic NAT

Network Address Translation (NAT) is the process where a network device, such as a Cisco router, assigns a public address to host devices inside a private network. The main reason to use NAT is to reduce the number of public IP addresses that an organization uses because the number of available public addresses is limited. This lab was adapted to run entirely on laptops, without Cisco hardware, by using previous Lab's concepts (Lab 0 and Lab 2). We use the same network set-up and requirements.

After DHCP is set up, NAT can be executed and it will enable dynamic NAT for both LAN1 and LAN2. After running, PC1 and PC2 should be able to reach the ISP namespace. The NAT process can be launched, only if these conditions are met:

```
#DHCP clients already have IPs from the server (check ip addr show dhcp_pc1 /  
dhcp_pc2).  
sudo ip netns exec PC1 ip addr show dhcp_pc1  
sudo ip netns exec PC2 ip addr show dhcp_pc2  
# Gateway namespace is configured with LAN and WAN interfaces.  
sudo ip netns exec GATEWAY ip addr show  
# IP forwarding is enabled on the gateway  
sudo ip netns exec GATEWAY sysctl -w net.ipv4.ip_forward=1  
  
#Test connectivity from PC1 and PC2 can be checked  
sudo ip netns exec PC1 ping -c 3 209.165.200.225  
sudo ip netns exec PC2 ping -c 3 209.165.200.225
```

Part 1: Configure and verify dynamic NAT (DNAT)

Dynamic NAT, in this lab, means that multiple private hosts (192.168.10.0/24, 192.168.20.0/24) could share one WAN IP: 209.165.200.226. The translation is automatic & dynamic (PAT / masquerade). ISP does NOT need static routes.

It is worth noting that PAT (Port Address Translation) is a type of dynamic NAT. Multiple private IPs can share one public IP. The router (gateway) tracks source ports to distinguish flows. All LAN hosts appear as one IP to the outside world, but the router uses different source ports to keep connections separate. For instance, in our example, the ISP sees two flows from the same IP but different ports, and the gateway keeps track of both flows to deliver replies correctly.

LAN	GATEWAY (PAT)	ISP (Maroc Telecom)
192.168.10.10:50000	→ 209.165.200.226:52311	→ 209.165.200.225:80
192.168.20.10:50000	→ 209.165.200.226:52312	→ 209.165.200.225:80

Step 1: Enable IP forwarding on the GATEWAY

```
sudo ip netns exec GATEWAY sysctl -w net.ipv4.ip_forward=1
```

Step 2: Flush existing iptables rules (clean state)

#Removes any old firewall or NAT rules. Default policy usually becomes ACCEPT if nothing was set.

```
sudo ip netns exec GATEWAY iptables -F
sudo ip netns exec GATEWAY iptables -t nat -F
sudo ip netns exec GATEWAY iptables -X
```

Step 3: Allow forwarding from LANs to WAN

Allows PC1 and PC2 to send traffic toward the ISP.

```
sudo ip netns exec GATEWAY iptables -A FORWARD -i gw1 -o veth3 -j ACCEPT
sudo ip netns exec GATEWAY iptables -A FORWARD -i gw2 -o veth3 -j ACCEPT
# Writing iptables FORWARD rules to allow traffic between these interfaces.
```

PC1 -- gw1 -- GATEWAY -- veth3 -- ISP

PC2 -- gw2 -- GATEWAY -- veth3 -- ISP

gw1, gw2 → interfaces connected to LANs

veth3 → interface connected to ISP/WAN

GATEWAY → doing NAT + forwarding

Step 4: Allow return traffic from WAN to LANs

Allows only replies to existing connections back into the LANs.

```
sudo ip netns exec GATEWAY iptables -A FORWARD -i veth3 -o gw1 -m state --state
ESTABLISHED,RELATED -j ACCEPT
sudo ip netns exec GATEWAY iptables -A FORWARD -i veth3 -o gw2 -m state --state
ESTABLISHED,RELATED -j ACCEPT
```

Step 5: Configure Dynamic NAT (MASQUERADE)

```
# This rewrites the source IP of packets leaving the WAN interface (veth3) to the gateway's WAN IP (209.165.200.226).
sudo ip netns exec GATEWAY iptables -t nat -A POSTROUTING -s 192.168.10.0/24 -o veth3 -j MASQUERADE
sudo ip netns exec GATEWAY iptables -t nat -A POSTROUTING -s 192.168.20.0/24 -o veth3 -j MASQUERADE
```

Step 6: Verify NAT configuration and test connectivity

```
# You should see packet counters increasing when traffic flows.
sudo ip netns exec GATEWAY iptables -t nat -L -v
# Test connectivity, Both pings should succeed, and the ISP will see traffic only from 209.165.200.226.
sudo ip netns exec PC1 ping 209.165.200.225
sudo ip netns exec PC2 ping 209.165.200.225
```

Step 6: Create a long-lived TCP connection

```
# Check if NAT + FORWARD rules are actually working.
# On ISP
sudo ip netns exec ISP nc -l 80
# On PC1
sudo ip netns exec PC1 nc 209.165.200.225 80
# On PC2
sudo ip netns exec PC2 nc 209.165.200.225 80
# Now check:
sudo ip netns exec GATEWAY conntrack -L
```

You will see something like:

```
tcp 6 431999 ESTABLISHED
src=192.168.10.10 dst=209.165.200.225 sport=44562 dport=80
src=209.165.200.225 dst=209.165.200.226 sport=80 dport=52311
tcp 6 431980 ESTABLISHED
src=192.168.20.10 dst=209.165.200.225 sport=34392 dport=80
src=209.165.200.225 dst=209.165.200.226 sport=80 dport=34392
```

Step 7: Create a Bash script to automate the NAT lab

Create a Bash script (**enable_dynamic_nat.sh**) containing the commands above to automate the configuration of dynamic NAT (MASQUERADE) on the GATEWAY, similar to how we automated network deployment. This script will make it easy to reuse the NAT setup in the future without needing to enter the commands manually. This will allow you execute the scripts separately (Lab 0, Lab 2, Lab 3).

Step 8: Bash script to clean and reset the NAT lab

Create a Bash script (**clean_dynamic_nat.sh**) to clean and reset the NAT lab environment so that the NAT setup can be safely re-run without rebuilding the entire virtual network (Lab 0) or launching the DHCP script (Lab 2). The cleanup script must automatically perform the following:

- Flush all filter rules (iptables -F) in the Gateway namespace.
- Flush all NAT rules (iptables -t nat -F) in the Gateway namespace.
- Reset default FORWARD chain policy to ACCEPT.
- Clear all existing connection tracking entries (conntrack -F).

Part 2: Configure and verify Static NAT (SNAT)

Unlike dynamic NAT / PAT, SNAT maps a specific private IP to a specific public IP, 1:1, without dynamic port translation. Static NAT uses a one-to-one mapping of local and global addresses, and these mappings remain constant. Static NAT is particularly useful for web servers or devices that must have static addresses that are accessible from the Internet.

Step 1: Static NAT (1:1) based on the following:

Internal host	Public IP
PC1 192.168.10.10	209.165.200.226
PC2 192.168.20.10	209.165.200.227

Step 2: Enable forwarding (mandatory)

```
sudo ip netns exec GATEWAY sysctl -w net.ipv4.ip_forward=1
```

Step 3: Cleanup (iptables only)

```
sudo ip netns exec GATEWAY iptables -F  
sudo ip netns exec GATEWAY iptables -t nat -F  
sudo ip netns exec GATEWAY iptables -X
```

Step 4: Configure Static NAT (SNAT)

```
# PC1 → Public IP  
sudo ip netns exec GATEWAY iptables -t nat -A POSTROUTING -s 192.168.10.10 -o veth3 -j SNAT -  
-to-source 209.165.200.226  
  
# PC2 → Public IP  
sudo ip netns exec GATEWAY iptables -t nat -A POSTROUTING -s 192.168.20.10 -o veth3 -j SNAT -  
-to-source 209.165.200.227
```

Step 5: Allow Forwarding in iptables

```
# LAN → WAN  
sudo ip netns exec GATEWAY iptables -A FORWARD -i gw1 -o veth3 -j ACCEPT  
sudo ip netns exec GATEWAY iptables -A FORWARD -i gw2 -o veth3 -j ACCEPT  
  
# WAN → LAN (for replies)  
sudo ip netns exec GATEWAY iptables -A FORWARD -i veth3 -o gw1 -m state --state  
ESTABLISHED,RELATED -j ACCEPT  
sudo ip netns exec GATEWAY iptables -A FORWARD -i veth3 -o gw2 -m state --state  
ESTABLISHED,RELATED -j ACCEPT
```

Step 6: Verification

```
#From PC1
sudo ip netns exec PC1 ping 209.165.200.225
sudo ip netns exec PC2 ping 209.165.200.225
```

PC1 uses 209.165.200.226, which is the GATEWAY's own WAN IP. So, the ISP already knows how to reach it. But, PC2 uses 209.165.200.227, which does NOT exist on any ISP interface, so the ISP has no idea where to send replies. Replies to PC2 are dropped at the ISP. To verify this, perform the following verify which public IP each PC is using.

```
# Run tcpdump on the ISP side
sudo ip netns exec ISP tcpdump -n -i isp0 icmp
# PC1, NATed to 209.165.200.226
sudo ip netns exec PC1 ping 209.165.200.225
# PC2, ICMP echo request, no echo reply, ISP does NOT know where 209.165.200.227 lives.
sudo ip netns exec PC2 ping 209.165.200.225
# Verify that conntrack is correct (NAT is NOT the issue) NAT is working, routing is broken on ISP
sudo ip netns exec GATEWAY conntrack -L
```

,

In order to fix it, a host route on the ISP for PC2's public IP must be added.

```
# GATEWAY default route to ISP
sudo ip netns exec GATEWAY ip route add default via 209.165.200.225 dev veth3
# ISP route back to PC2 public IP
sudo ip netns exec ISP ip route add 209.165.200.227/32 via 209.165.200.226 dev isp1
```

Step 7: Verification

```
#From PC1
sudo ip netns exec PC1 ping 209.165.200.225
sudo ip netns exec PC2 ping 209.165.200.225
#On WAN
sudo ip netns exec GATEWAY tcpdump -i veth3 -n
```

Step 8: Bash script to automate the SNAT lab

Develop a Bash script (**enable_static_nat.sh**) that automates the configuration of Static NAT for PC1 and PC2 using the abovementioned commands. The script should include:

- Enabling IP forwarding on the GATEWAY namespace.
- Flushing existing iptables rules and clearing NAT rules on the GATEWAY.
- Adding the SNAT rules to map the private LAN IPs to the assigned public IPs.
- Configuring the necessary FORWARD rules on the GATEWAY.
- Adding the appropriate static route on the ISP namespace for the second PC.

By implementing a script, you will avoid executing all commands manually step by step, ensure consistency, and make it easy to reuse labs for future.

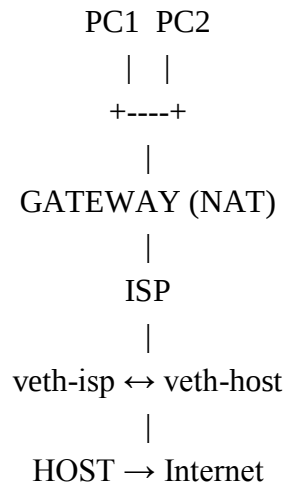
Step 9: Bash script to clean and reset the Static NAT lab

Create a Bash script (**clean_static_nat.sh**) to clean and reset the Static NAT lab environment so that the Static NAT configuration can be safely re-run without rebuilding the entire virtual network (Lab 0).

PART 3: Connect ISP to HOST Internet

This part is responsible for allowing the ISP namespace to route traffic from the lab network to the real Internet. The focus will be on static NAT according to the following figure:

- PCs send traffic to GATEWAY, which NATs it to lab public IPs.
- GATEWAY forwards to ISP namespace, which NATs traffic to HOST.
- HOST sends packets to the real Internet (8.8.8.8).
- Lab DNS queries go to DNS namespace, external domains are forwarded to 8.8.8.8.



Step 1: Create a veth pair between ISP and HOST

veth-isp goes into the ISP namespace. veth-host stays in the host (Linux machine) to provide access to the real Internet.

```
sudo ip link add veth-isp type veth peer name veth-host || true
sudo ip link set veth-isp netns ISP
```

Step 2: Assign IP addresses

ISP side gets 10.0.0.2, host side 10.0.0.1.

```
sudo ip netns exec ISP ip addr add 10.0.0.2/30 dev veth-isp
sudo ip netns exec ISP ip link set veth-isp up
sudo ip addr add 10.0.0.1/30 dev veth-host
sudo ip link set veth-host up
```

Step 3: Set default route in ISP

All traffic from the ISP namespace that is not destined for lab networks goes to the host (10.0.0.1).

```
sudo ip netns exec ISP ip route replace default via 10.0.0.1
```

Step 4: Enable IP forwarding in ISP


```
# Allows packets coming into the ISP namespace to be forwarded to another interface (veth-isp → Internet).
sudo ip netns exec ISP sysctl -w net.ipv4.ip_forward=1
```

Step 5: Configure NAT in ISP

```
# Flush old NAT rules (-F) to avoid conflicts. FORWARD rules:
# Allow traffic from lab network → ISP → host (isp1 → veth-isp).
# Allow responses back (veth-isp → isp1) only for established connections.
# MASQUERADE NAT hides lab IPs behind veth-isp IP (10.0.0.2) so host can route packets to the real Internet.
sudo ip netns exec ISP iptables -t nat -F
sudo ip netns exec ISP iptables -F
sudo ip netns exec ISP iptables -A FORWARD -i isp1 -o veth-isp -j ACCEPT
sudo ip netns exec ISP iptables -A FORWARD -i veth-isp -o isp1 -m state --state ESTABLISHED,RELATED -j ACCEPT
sudo ip netns exec ISP iptables -t nat -A POSTROUTING -o veth-isp -j MASQUERADE
```

Step 6: Enable NAT and forwarding on HOST

```
# IP forwarding allows host to send packets from lab to Internet.
sudo sysctl -w net.ipv4.ip_forward=1

# NAT (MASQUERADE) hides the ISP subnet (10.0.0.0/30) behind the host's real Internet IP.
# $HOST_IF automatically selects the host interface connected to Internet (e.g., eth0).
HOST_IF=$(ip route | awk '/default/ {print $5}')
sudo iptables -t nat -A POSTROUTING -s 10.0.0.0/30 -o "$HOST_IF" -j MASQUERADE

# PCs in lab → GATEWAY → ISP → HOST → Internet.
# PC1 & PC2 can now ping external IPs (8.8.8.8) and resolve external domains if DNS is set.
```

Step 7: Update scripts and re-run them with verification checks

The scripts will be used for future Labs, so make sure to have them ready for execution to avoid manual execution of Lab 0, Lab 2 and Lab 3.

Deliverables (To be uploaded on the Platform)

In this lab, you must design and configure a Linux-based gateway using network namespaces to provide NAT connectivity for two LAN hosts. Both dynamic NAT (MASQUERADE, DNAT) and static NAT (SNAT) need to be implemented and tested, enabling the hosts to communicate with an external network (ISP namespace) and Internet as well. The main report document (word/pdf, 10 pages Max) must include the following:

- NAT configuration
 - iptables rules used for dynamic NAT (MASQUERADE) and static NAT (SNAT)
 - Explanation of how NAT translates private IPs to public IPs in this topology
- Verification and testing
 - Ping tests from PC1 and PC2 to external addresses (ISP) and Internet (8.8.8.8)
 - Routing table verification for each namespace
 - iptables rule counters (iptables -L -v -n)
 - Packet capture evidence (tcpdump) showing NAT translation and forwarded packets
- Conclusion
 - Summary of achieved objectives (successful NAT and forwarding)
 - Short reflection on the role of NAT and the use of network namespaces in Linux for network emulation

A single compressed file named in the format “lastname.firstname-Lab3.zip”, must be uploaded on the platform (see deadlines) and must include:

- The main report document (Word or PDF)
- All developed Bash scripts